

2025 SP Assignment #1

Due: Oct. 22nd 12:00pm

Last updated: 2025-09-30

1. Assignment Overview

The Linux kernel provides an interface through a mechanism called a Kernel Module, which allows for the dynamic addition or removal of functionalities without recompiling or rebooting the running kernel. Furthermore, the `proc` virtual file system serves as a channel to easily deliver various internal kernel data structures and statistical information to the User Space in text format.

In this assignment, you will implement a kernel module that can directly query the internal state of the process and scheduler, which are core data structures of the kernel, through kernel module programming, and verify it through the `proc` virtual file system. The goal is to gain practical experience of how the processes and schedulers learned in class are implemented in Linux. The assignment consists of a report and a programming assignment.

2. Background: Kernel Module Programming

This programming assignment involves developing a kernel module that uses kernel module programming to check desired kernel information via the `proc` file system. A kernel module program can be dynamically added to the kernel to operate when needed, without directly modifying and compiling the Linux kernel code. Since a kernel module program operates as part of the kernel, it can be involved in the kernel's operations and access internal kernel data structures. The `proc` file system is one of Linux's virtual file systems, allowing users to easily obtain information from the kernel. For example, a user can get system memory information by typing `cat /proc/meminfo` in the terminal. This assignment will also use this method to obtain necessary system information. Such a `proc` file system can be implemented through kernel module programming. In this assignment, you will not modify the kernel code directly.

An example code for a kernel module program that displays the message "Hello, Kernel!" when a user types `cat /proc/hello` in the terminal is attached with this document. You can refer to the example code when working on this assignment. Note that kernel module programs must use a separate compiler and headers instead of the standard `gcc` compiler and standard libraries. In other words, the C language rules for kernel modules have parts that are different from standard C written for the `gcc` compiler. The Makefile rules are also somewhat different, which can be also checked at the provided code.

A kernel module is registered with the kernel using the `sudo insmod` command in the terminal, at which point the function registered via `module_init()` is executed. A kernel module is removed from the kernel using the `sudo rmmod` command in the terminal, and the function registered via `module_exit()` is executed. This basic structure must be followed when programming kernel modules. When programming kernel modules, it is recommended to use the `seq_printf()` function for output, as shown in the example code. Also, the `proc_ops` structure is used to create entries in the `proc` file system. For detailed usage, refer to the attached kernel module programming guide site.

3. Report Assignment

3.1 Report Format

The report consists of a **preliminary research** section and a **programming results** section. This assignment requires you to use the provided ACM template, which is a standard format used in academic papers. It helps structure the logical flow, experimental design, and analysis of results. The detailed requirements are as follows:

- The report title should be “OS Assignment 2 Report”.
- Section 1 of the report should contain the preliminary research. The subsequent sections should detail the programming results.
- Section titles (1, 2, 3, ...) should be in 10pt font. Section subtitles (1.1, 1.2, 2.1, ...) and the main text should be in 9pt.
- Use “Linux Libertine O” font for English text and “NanumSquare” font for Korean text.
- You may use any language (Korean or English) for the report text.
- Follow all other formatting rules in the provided template.

3.2 Report Contents

- **Preliminary Research Section**

- Compare and analyze the Linux kernel code of the following two versions:
 - Scheduler-related code of Linux kernel version 2.6.39
 - Scheduler-related code of Linux kernel version 6.14
- Compare and analyze the scheduling-related kernel data structures for each version.
 - Focus the analysis on `cfs_rq`, `sched_entity`, and `task_struct`.
 - Analyze member variables of each kernel data structure that are important from a scheduling perspective or related to the programming assignment.
 - Compare and analyze focusing on the common grounds and differences between the versions.
- Analyze how the CFS scheduler selects the next process from the Red-Black Tree.
- You can refer to the L02 lecture notes.
- The Linux kernel code needed for code analysis can be found at the link below.
 - <https://elixir.bootlin.com/linux/v6.17/source>
- Use flowcharts or diagrams to organize the information (3 or more).
- **References**

- **Programming Results Section**

- The overall structure and operation process of the created kernel modules.
 - Describe each of the 2 modules implemented in the programming assignment separately.
 - **Must use diagrams such as flowcharts or block diagrams.**
 - Focus on the overall flow rather than detailed code explanations.
- Specify the development environment.
 - `uname -a` command output
 - Compiler version, memory information, etc.
- Difficulties encountered during the assignment and how you resolved.

4. Programming Assignments

The programming assignment is to write a total of 2 kernel modules. The preparations for the programming assignment and detailed descriptions for each module are as follows.

4.1 Prerequisites

- It is recommended to carry out the assignment in a virtual machine environment like VirtualBox or VMWare, as the kernel may crash during the process. **The number of CPU cores for the virtual machine must be set to 2 or more.**
- Download the 64-bit PC image **Ubuntu-24.04.3** and install it in the virtual machine environment. Note that the grading of the programming assignment will be based on whether the submitted program operates correctly on Ubuntu version 24.04.3. When Ubuntu 24.04.3 is installed, the kernel version will be 6.14.0 (6.14.0-32-generic) (verifiable with `uname -a`). If the kernel version is not 6.14.0, you must change the kernel version to 6.14.0. You can download the corresponding kernel from the links below.
 - <https://releases.ubuntu.com/noble/>
 - <https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.14.tar.gz>

4.2 PART 1 – ancestor

This module takes a specific PID as input and outputs information about all ancestor processes of that process. The proc file created by the kernel module should be named "ancestor".

- You can check the desired information from the terminal with the following command:

```
$ cat /proc/ancestor
```

- The module must create a readable/writable file named `/proc/ancestor`
 - **Write operation:** The `echo <PID> > /proc/ancestor` command should store the PID entered by the user inside the module.
 - **Read operation:** The `cat /proc/ancestor` command should print the information of all ancestors, starting from the process with the stored PID, one line at a time.
- The output format and an example are as follows.
 - First, print the basic information at the top.
 - Student ID, Name
 - Then, print the ancestor processes one by one, starting from the specified PID process.
 - **Be careful about the output format, as deductions may occur if it differs.**
 - The number of '-' characters between the basic information and the process information is 40.

```
// Output format: [PID] <process name>
```

```
ID: 2025123456
```

```
Name: Hong, Gildong
```

```
-----
```

```
[5432] bash
```

```
[5401] gnome-terminal
```

```
[1] systemd
```

4.3 PART 2 - proc_analyzer

This module takes a process PID as input and displays detailed scheduling information, categorized by CPU, for only those processes that are in the CFS runqueue among the specified process and all its descendants.

- The module must create a readable/writable file named /proc/proc_analyzer
 - **Write operation:** The echo <PID> > /proc/proc_analyzer command should store the user-input PID inside the module.
 - **Read operation:** The cat /proc/proc_analyzer command should search for the stored PID and all its descendants. Among them, only the processes currently waiting to be executed on the CFS runqueue should be grouped by CPU, and within each group, they should be printed in **ascending order of their vruntime**.
- The output format and an example are as follows. (Note that the values below are arbitrary examples.)
 - First, print the basic information at the top, same as in PART 1.
 - Additionally, print the input PID as shown in the example.
 - Then, for each CPU, print the number of processes in the CFS runqueue and the processes themselves in **ascending order of vruntime**.
 - CPU numbers start from 0 and should be printed in **ascending order**.
- In a standard Ubuntu environment where no special programs are running, there are typically only one or two processes registered in the CFS runqueue at any given moment. Therefore, you can implement and run an arbitrary test program that creates and runs multiple processes to test your implementation.
- Appropriate locking may be required when accessing kernel data.

```
// Output format: [PID] <process name> <vruntime>
```

```
ID: 2025123456
```

```
Name: Hong, Gildong
```

```
PID: 3734
```

```
-----
```

```
[CPU #0] Running processes: 2
```

```
[3286] python3 123456439045
```

```
[3290] python3 123456462280
```

```
-----
```

```
[CPU #1] Running processes: 3
```

```
[3391] cat 123457185643
```

```
[3280] python3 123458242690
```

```
[3289] python3 124578976015
```

4.4 Programming Assignment Compilation Requirements

You must use **one Makefile** to compile both parts. That is, when the build is initiated with the make command, the kernel module files ancestor.ko and proc_analyzer.ko should be created in the same directory where the command was executed. make clean should delete all files generated by the build. The created kernel modules must be registerable with the sudo insmod command and removable with the sudo rmmod command.

(example)

```
os@ubuntu:~$ cd 2025123123
```

```
os@ubuntu:~/2025123123$ make
```

```
os@ubuntu:~/2025123123$ ls
Makefile ancestor.ko proc_analyzer.ko ...
os@ubuntu:~/2025123123$ make clean
Makefile ...
```

5. Assignment Submission

You must submit **two** items: 1) A **report file** named after your student ID, converted to PDF (e.g., hw1_2025123123.pdf) and 2) A **source code archive** (tar file) named after your student ID. Regarding the **report file**, The filename must be exactly your student ID, and it must be in PDF format. Regarding the source code, you must create archive file using the commands below. **No other compression format or double compression is allowed** and **does not include your PDF report inside this tar file**.

```
# Assume you are working in the directory ~/hw1, and that your Makefile is in ~/hw1/.
(vi ~/hw1/Makefile)
# Suppose your student ID is 2025123123. After you finish everything, run the following commands to package
your work and submit hw1_2025123123.tar.
# This means that when hw1_2025123123.tar is extracted, there should be exactly one folder named
2025123123. Inside it, the Makefile should be directly visible (no extra subdirectories).
$ cd ~/hw1/..
$ mv hw1 2025123123
$ tar cvf hw1_2025123123.tar 2025123123
```

6. Precautions

- If the assignment does not compile in the grading environment, the programming part will receive 0 points, so pay attention to the Linux distribution version, file encoding, kernel version, etc.
- The number of CPUs will be arbitrarily set to 2 or more, so the code must be able to run in various CPU environments.
- **No leniency is given for unreasonable delay—late submissions are not accepted.**
- All references (including images and code snippets) **must be cited** (lack of citation results in point deductions).
- If you copy or modify another student's work, **both** parties get zero points.
- Failure to follow the submission file generation instructions or to match the exact structure (**especially regarding filenames**) may lead to point deductions or a zero score.
- **Add comments to your program codes** (If not, the points will be deducted).
- Submissions that **fail to compile** receive zero points.
- For assignment-related questions, post a **public inquiry** on the LearnUs Q&A board.

7. References

- <https://elixir.bootlin.com/linux/v6.14/source>
- <https://sysprog21.github.io/lkmpg/>
- <https://docs.kernel.org/core-api/index.html>